

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

База знаний Prolog и онтология видеоигр
по курсу «Системы искусственного интеллекта»

Модуль 1. Базы знаний и онтологии

Выполнили:

Якименко Владислав, Р3313

Пахомов Игнат, Р3313

Проверил:

Жданов Андрей Дмитриевич

Содержание

1	Цель и требования	1
2	База знаний Prolog	1
2.1	Предметная область и словарь	1
2.2	Правила и порядок вычисления	1
2.3	Отсечение и полнота	2
2.4	Фактические ответы интерпретатора	2
3	Онтология в Protégé	4
3.1	Перевод фактов и правил	4
3.2	Иерархия и свойства	5
3.3	Открытый мир и SWRL	5
3.4	Компетентностные запросы и проверка	5
4	Тестирование и вывод	6
5	Источники	6

1 Цель и требования

Создать базу знаний об играх и перевести её в онтологическую форму. Требования: не менее 20 унарных фактов, 10–15 бинарных фактов, 5–7 правил, запросы с переменными и логическими операциями. Для онтологии нужны классы, свойства, индивиды, иерархия, ограничения и проверка вывода. Работа включает обе части лабораторной 1 по выданному заданию.

2 База знаний Prolog

2.1 Предметная область и словарь

Объекты представлены атомами в `snake_case`: например, `portal_2`, `valve`. Переменные начинаются с заглавной буквы. Факт утверждает отношение, а правило задаёт условия его доказательства. Запятая означает конъюнкцию, точка с запятой — дизъюнкцию. Отрицание `\+ Goal` успешно, когда интерпретатор не смог доказать цель `Goal`.

Таблица 1: Факты исходной базы

Предикат	Число фактов
<code>game/1</code>	12
<code>single_player/1</code>	11
<code>multiplayer/1</code>	6
<code>story_driven/1</code>	7
<code>difficult/1</code>	5
<code>indie/1</code>	4
<code>relaxing/1</code>	3
<code>competitive/1</code>	2
<code>free_to_play/1</code>	1
Всего унарных	51
<code>developed_by/2</code>	10
<code>has_genre/2</code>	5

2.2 Правила и порядок вычисления

Семь правил описывают сюжетные одиночные игры, дружелюбные многопользовательские, напряжённые, сюжетные независимые, связь игры со студией, спокойные одиночные и подходящие новичку игры. Пример композиции условий:

```
recommended_for_beginner(Game) :-  
    game(Game), single_player(Game),  
    \+ difficult(Game), \+ competitive(Game).
```

Сначала выбирается конкретная игра, затем проверяются отрицания. Такая последовательность существенна: проверка отрицания со свободной переменной спрашивала бы об отсутствии любой сложной игры, а не о свойствах кандидата. В `intense_game/1` игра Civilization VI удовлетворяет двум ветвям ИЛИ; `setof/3` устраняет повторения в итоговом наборе.

2.3 Отсечение и полнота

Предикат `!` отсекает альтернативы, возникшие до него в пределах текущего вызова. В запросе `story_driven(G), !, \+ developed_by(G,cd_projekt_red)` первым выбирается The Witcher 3. Отрицание не выполняется, а возвращаться к следующим сюжетным играм уже нельзя, поэтому ответ — `false`. Без отсечения находятся Portal 2, Hades, Hollow Knight, Celeste и Baldur's Gate 3. Здесь `cut` меняет множество решений и не подходит для получения всех рекомендаций.

2.4 Фактические ответы интерпретатора

Ниже приведены десять запросов, выполненных скриптом сбора результатов. `findall/3` собирает ответы в порядке поиска; `setof/3` возвращает отсортированный набор. Списки совпадают с отдельными интерактивными решениями.

Запрос 1

```
?- game(minecraft).  
true
```

Запрос 2

```
?- developed_by(portal_2,valve).  
true
```

Запрос 3

```
?- findall(G,developed_by(G,valve),L).  
[portal_2, counter_strike_2]
```

Запрос 4

```
?- findall(G,(single_player(G),story_driven(G)),L).  
[the_witcher_3, cyberpunk_2077, portal_2, hades, hollow_knight,  
celeste, baldurs_gate_3]
```

Запрос 5

```
?- findall(G,(game(G),(relaxing(G);free_to_play(G)),\+ competitive(G)),L).  
[minecraft, stardew_valley]
```

Запрос 6

```
?- findall(G,friendly_multiplayer(G),L).  
[portal_2, minecraft, stardew_valley, baldurs_gate_3]
```

Запрос 7

```
?- setof(G,intense_game(G),L).  
[celeste, civilization_vi, counter_strike_2, doom_eternal, hades,  
hollow_knight]
```

Запрос 8

```
?- findall(G,indie_gem(G),L).  
[hades, hollow_knight, celeste]
```

Запрос 9

```
?- findall(G,recommended_for_beginner(G),L).  
[the_witcher_3, cyberpunk_2077, portal_2, minecraft, stardew_valley,  
baldurs_gate_3]
```

Запрос 10

```
?- findall(G,relaxing_solo_game(G),L).  
[minecraft, stardew_valley, civilization_vi]
```

3 Онтология в Protégé

3.1 Перевод фактов и правил

Онтология содержит 21 именованный класс, 24 именованных индивида, три объектных свойства, одно свойство данных, семь эквивалентностей и одно правило SWRL. Из 24 индивидов 12 являются играми, 8 — студиями и 4 — жанрами. Пространство имён: <http://example.org/itmo/games#>. OWL и Turtle представляют один RDF-граф из 329 троек; их изоморфность проверяется скриптом.

Таблица 2: Трассируемость Prolog и OWL

Prolog	OWL
<code>game(G)</code>	принадлежность <code>VideoGame</code>
<code>single_player(G), multiplayer(G)</code>	<code>SinglePlayerGame</code> , <code>MultiplayerGame</code>
<code>story_driven(G), difficult(G)</code>	<code>StoryDrivenGame</code> , <code>DifficultGame</code>
<code>indie(G), relaxing(G)</code>	<code>IndieGame</code> , <code>RelaxingGame</code>
<code>competitive(G), free_to_play(G)</code>	<code>CompetitiveGame</code> , <code>FreeToPlayGame</code>
<code>developed_by(G,S)</code>	<code>developedBy(G,S)</code> , обратное <code>developedGame(S,G)</code>
<code>has_genre(G,T)</code>	<code>hasGenre(G,T)</code>
<code>solo_story_game(G)</code>	<code>SoloStoryGame</code> : <code>SinglePlayerGame</code> and <code>StoryDrivenGame</code>
<code>friendly_multiplayer(G)</code>	<code>MultiplayerGame</code> and <code>NonCompetitiveGame</code>
<code>intense_game(G)</code>	<code>DifficultGame</code> or <code>CompetitiveGame</code>
<code>indie_gem(G)</code>	<code>IndieGame</code> and <code>StoryDrivenGame</code>
<code>relaxing_solo_game(G)</code>	<code>RelaxingGame</code> and <code>SinglePlayerGame</code>
<code>recommended_for_beginner(G)</code>	<code>SinglePlayerGame</code> and <code>NonDifficultGame</code> and <code>NonCompetitiveGame</code>
<code>studio_game(G,S)</code>	<code>StudioGame</code> — проекция на <code>G</code> ; <code>S</code> извлекается через <code>developedBy</code>

Двухаргументный предикат нельзя полностью заменить одним классом: `StudioGame` показывает наличие разработчика, но для получения пары игра–студия необходимо объектное свойство. В исходном наборе десять явно известных таких пар. По семи остальным унарным результатам, включая отдельную проверку SWRL, множества экземпляров сопоставляются с результатами Prolog автоматически.

3.2 Иерархия и свойства

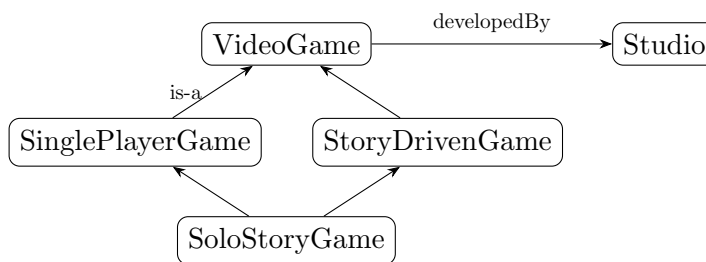


Рис. 1: Фрагмент иерархии и связи между играми и студиями

Классы `VideoGame`, `Studio` и `Genre` попарно несовместимы. При этом `SinglePlayerGame` и `MultiplayerGame` совместимы: `Portal 2` относится к обоим. Domain свойства `developedBy` равен `VideoGame`, range — `Studio`; у `developedBy` направления обратные. У `hasGenre` domain `VideoGame`, range `Genre`. Domain и range служат аксиомами вывода типов, а не проверками ввода в стиле реляционной базы.

Добавлен `DataProperty title` с диапазоном `xsd:string`. У `VideoGame` задано `title exactly 1 xsd:string`, у `StudioGame` — `developedBy min 1 Studio`. Названия являются дополнительными данными отображения относительно исходного Prolog. Минимальная кардинальность в OWL допускает неизвестный объект; отсутствие явно записанного значения само по себе не делает онтологию противоречивой. В отличие от этого, два разных строковых `title` нарушают `exactly 1`.

3.3 Открытый мир и SWRL

Prolog трактует отсутствие доказательства как неудачу цели. OWL не выводит отрицание из отсутствия факта. Поэтому для данного конечного набора явно заданы `NonCompetitiveGame` и `NonDifficultGame`, несовместимые с положительными категориями. Эти утверждения добавлены при переводе и требуют пересмотра при изменении набора фактов.

Для отдельной демонстрации SWRL задано правило:

```
SinglePlayerGame(?g) ^ StoryDrivenGame(?g)
-> RuleVerifiedStoryGame(?g)
```

Этот класс не имеет `Equivalent To`, поэтому его семь экземпляров демонстрируют именно срабатывание правила. HermiT проверил правило для именованных индивидов. Остальные логические определения выражены штатными конструкциями OWL.

3.4 Компетентностные запросы и проверка

В Protégé файл открывается через `File → Open`. После запуска `Reasoner → HermiT → Start reasoner` на вкладке `DL Query` выбирается `Instances` и вводятся имена ниже. При отображении `prefixed names` используется ведущий символ двоеточия.

1. `SoloStoryGame`: `baldurs_gate_3`, `celeste`, `cyberpunk_2077`, `hades`, `hollow_knight`, `portal_2`, `the_witcher_3`.
2. `FriendlyMultiplayerGame`: `baldurs_gate_3`, `minecraft`, `portal_2`, `stardew_valley`.
3. `IntenseGame`: `celeste`, `civilization_vi`, `counter_strike_2`, `doom_eternal`, `hades`, `hollow_knight`.
4. `IndieGem`: `celeste`, `hades`, `hollow_knight`.
5. `BeginnerRecommendedGame`: `baldurs_gate_3`, `cyberpunk_2077`, `minecraft`, `portal_2`, `stardew_valley`, `the_witcher_3`.
6. `RuleVerifiedStoryGame`: тот же набор из семи игр, что у `SoloStoryGame`.

Наборы выше получены HerMiT в автоматическом запуске, а не переписаны из предполагаемого результата. Файл `verification.json` фиксирует вывод. Для визуального просмотра можно включить OntoGraf и раскрыть `VideoGame`; рисунок в отчёте показывает фрагмент тех же отношений. Дополнительно SPARQL с `FILTER NOT EXISTS` выбирает сюжетные игры без записанной связи с `Valve`: `baldurs_gate_3`, `celeste`, `cyberpunk_2077`, `hades`, `hollow_knight`, `the_witcher_3`. Это проверка отсутствия тройки в графе, а не OWL-доказательство отрицания. Запрос `DL not (developedBy value valve)` в общем случае не эквивалентен такому фильтру.

4 Тестирование и вывод

Все 13 тестов Prolog прошли в SWI-Prolog 10.0.2. Выполнены десять запросов, приведённых в листингах. HerMiT подтвердил консистентность онтологии, семь наборов выведенных типов и обратное свойство. Негативные проверки `disjoint`, типа данных и кардинальности обнаружили противоречия.

Исходная БЗ удовлетворяла требованиям по объёму, но онтология нуждалась в добавлении `DataProperty`, кардинальностей и `SWRL`. Эти дополнения выполнены. Свойства игр приняты как учебные факты; жанровые данные неполны. Отрицательные классы OWL заданы явно для согласования с конечной Prolog-базой. При расширении базы их необходимо актуализировать.

В результате получена воспроизводимая модель предметной области, пригодная для последующего использования в рекомендательной системе. Команды запуска, запросы и протокол проверки включены в папки `lab_1` и `reports`.

5 Источники

Методическое пособие «Системы искусственного интеллекта», ИТМО, 2024, с. 56–57, приложение 2: <https://books.ifmo.ru/file/pdf/3308.pdf>. Задание «Системы ИИ.docx», лабораторная 1, части 1 и 2.